

AD COMPUTER SEARCH

Technische Umsetzungsbeschreibung

Architektur, Komponenten, Abläufe, Schnittstellen, Build und Tests

Projekt	ADComputerSearchTool
Dokumentstatus	Refaktoriierter Stand / Version 2.0
Technologie	C#, .NET 8, WPF, MVVM, LDAP, ClosedXML
Stand	18. August 2026

Erstellt für die interne Dokumentation und Weiterentwicklung der Anwendung.

Inhalt

- 1 Technischer Überblick
- 2 Lösungs- und Projektstruktur
- 3 Architektur und Abhängigkeiten
- 4 Datenmodell
- 5 Active Directory und DNS
- 6 MVVM, Commands und UI
- 7 Export, Clipboard und Dialoge
- 8 Dependency Injection und Start
- 9 Fehlerbehandlung und Sicherheit
- 10 Unit-Tests, Build und Deployment
- 11 Wartung und Erweiterung

1. Technischer Überblick

ADComputerSearchTool ist eine WPF-Desktopanwendung auf Basis von .NET 8 und C#. Die Anwendung verwendet LDAP über System.DirectoryServices.Protocols, DNS über System.Net, ClosedXML für den XLSX-Export sowie Microsoft.Extensions.DependencyInjection zur zentralen Auflösung der Komponenten.

Architekturprinzip

Die Anwendung folgt MVVM und Dependency Injection. Views enthalten nur UI-spezifisches Verhalten, ViewModels steuern Zustand und Commands, Services kapseln technische Zugriffe, Models repräsentieren Daten, und Helpers enthalten reine Transformationslogik.

Technologie	Verwendung
.NET 8 / C#	Laufzeit, Sprache und asynchrone Verarbeitung.
WPF / XAML	Desktopoberfläche, Datenbindung und DataGrid.
MVVM	Trennung von Darstellung, Zustand und Ablaufsteuerung.
LDAP	Lesender Zugriff auf Computerobjekte im Active Directory.
DNS	Auflösung des Hostnamens zu einer IPv4-Adresse.
ClosedXML	Erzeugung formatierter Excel-Arbeitsmappen.
Microsoft DI	Registrierung und Konstruktorinjektion der Komponenten.
xUnit	Automatisierte Unit-Tests.

2. Lösungs- und Projektstruktur

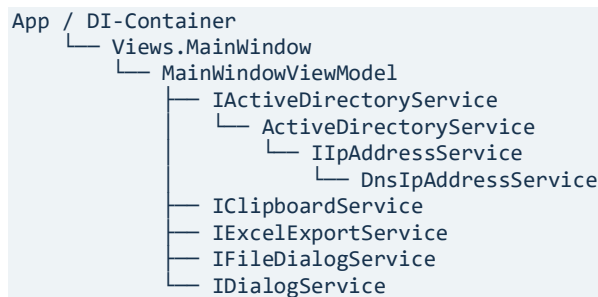
```
ADComputerSearchTool
├── Assets
├── Commands
├── Exceptions
├── Helpers
├── Models
├── Services
│   └── Interfaces
├── ViewModels
├── Views
├── App.xaml
├── App.xaml.cs
└── ADComputerSearchTool.csproj

ADComputerSearchTool.Tests
├── Helpers
├── Models
├── ViewModels
└── ADComputerSearchTool.Tests.csproj
```

Bereich	Verantwortung
---------	---------------

Bereich	Verantwortung
Assets	App-Icon und weitere statische Ressourcen.
Commands	RelayCommand und AsyncRelayCommand für ICommand.
Exceptions	Fachspezifische Ausnahmen für AD-Abfragen.
Helpers	LDAP-Filter, DN-Auswertung und WPF-Visual-Tree-Helfer.
Models	ComputerRecord, Suchkriterien und Status-Enum.
Services	Technische Implementierungen für AD, DNS, Export und UI-nahe Systemdienste.
Interfaces	Abstraktionen für lose Kopplung und Tests.
ViewModels	Zustand, Commands und Ablaufsteuerung.
Views	XAML und ausschließlich UI-spezifische DataGrid-Interaktion.
Tests	Automatisierte Tests für reine Logik und ViewModel-Verhalten.

3. Architektur und Abhängigkeiten



Die Abhängigkeiten werden ausschließlich über Konstruktoren injiziert. Das Hauptfenster kennt nur das MainWindowViewModel. Das ViewModel kennt ausschließlich Service-Interfaces. Konkrete Implementierungen werden zentral in App.xaml.cs registriert.

3.1 Lebensdauer der Komponenten

Komponente	Empfohlene Lebensdauer	Begründung
Technische Services	Singleton	Zustandsarm; eine Instanz reicht für die Anwendung.
MainWindowViewModel	Singleton	Ein Hauptfenster und ein konsistenter UI-Zustand.
MainWindow	Singleton	Die Anwendung besitzt ein Hauptfenster.

4. Datenmodell

4.1 ComputerRecord

ComputerRecord ist ein unveränderlich gedachtes Ergebnisobjekt. Die Eigenschaften werden bei der Erstellung initialisiert. Der sichtbare Status wird aus IsEnabled abgeleitet, sodass kein widersprüchlicher Zustand zwischen booleschem Wert und Text entstehen kann.

```
public sealed class ComputerRecord
{
    public string Name { get; init; } = string.Empty;
    public string IPAddress { get; init; } = string.Empty;
    public bool IsEnabled { get; init; }
    public string Status => IsEnabled ? "Aktiv" : "Deaktiviert";
    public DateTime? LastLogon { get; init; }
    // weitere Anzeigeeigenschaften ...
}
```

4.2 ComputerSearchCriteria

ComputerSearchCriteria bündelt alle Filterwerte. ComputerStatusFilter ist ein Enum mit All, Enabled und Disabled. Dadurch entfallen fehleranfällige Stringvergleiche in der Fachlogik.

5. Active Directory und DNS

5.1 LDAP-Verbindung

- Domänenname aus IPGlobalProperties.GetIPGlobalProperties().DomainName.
- LDAP-Port 389 mit AuthType.Negotiate.
- Windows-Anmeldeinformationen über CredentialCache.DefaultNetworkCredentials.
- LDAP-Protokollversion 3, Signing und Sealing aktiviert.
- Ermittlung des defaultNamingContext über eine Base-Scope-Abfrage.

5.2 LDAP-Suchfilter

LdapFilterHelper erstellt den Filter aus Computernamen, Beschreibung und Status. Sonderzeichen werden gemäß LDAP-Filteranforderungen maskiert. Der Basisfilter lautet (objectCategory=computer). Für den Status wird die bitweise Matching Rule 1.2.840.113556.1.4.803 auf userAccountControl verwendet.

```
All:      (&(objectCategory=computer))
Enabled:  (&(objectCategory=computer)
          (!(userAccountControl:1.2.840.113556.1.4.803:=2)))
Disabled: (&(objectCategory=computer)
          (userAccountControl:1.2.840.113556.1.4.803:=2))
```

5.3 Abgefragte Attribute

Attribut	Verarbeitung
name	Computernamen.
description	Beschreibung.

Attribut	Verarbeitung
distinguishedName	OU-Pfad sowie interne Eindeutigkeit.
userAccountControl	Ermittlung Aktiv/Deaktiviert.
memberOf	Auflösung der Gruppennamen.
lastLogonTimestamp	Konvertierung von Windows File Time in lokale Zeit.
operatingSystem	Betriebssystemanzeige.
dNSHostName	Hostname und Grundlage der DNS-Auflösung.

5.4 Distinguished Names

DistinguishedNameHelper zerlegt einen DN, ohne maskierte Kommata als Trenner zu interpretieren. OU-Werte werden als lesbarer Pfad verbunden. Bei Gruppenmitgliedschaften wird der erste CN-Bestandteil extrahiert und maskierte Zeichen werden zurückgewandelt.

5.5 DNS-Auflösung

DnsIpAddressService verwendet bevorzugt den AD-Wert dNSHostName und ersatzweise den Computernamen. Von den zurückgegebenen Adressen wird die erste IPv4-Adresse gewählt. Socket- und Namensauflösungsfehler ergeben eine leere Adresse, ohne die gesamte Suche abubrechen.

6. MVVM, Commands und UI

6.1 MainWindowViewModel

- ObservableCollection<ComputerRecord> für die Ergebnisliste.
- Bindbare Filtereigenschaften und SelectedStatus.
- SelectedComputer für die aktuelle Auswahl.
- IsBusy und IsNotBusy zur Steuerung von Bedienbarkeit und Fortschrittsanzeige.
- StatusMessage und ResultCountText für Rückmeldungen.
- SearchCommand, ClearCommand, ExportCommand und CopyRowCommand.

6.2 Commands

RelayCommand kapselt synchrone Aktionen. AsyncRelayCommand verhindert parallele Ausführung desselben Suchbefehls und aktualisiert CanExecute während der Verarbeitung. Das ViewModel stößt RaiseCanExecuteChanged an, wenn sich Busy-Status oder Ergebnisanzahl ändern.

6.3 XAML-Bindings

```
<TextBox Text="{Binding ComputerNameFilter,
    UpdateSourceTrigger=PropertyChanged}" />
<Button Content="Suchen" Command="{Binding SearchCommand}" />
<DataGrid ItemsSource="{Binding Results}"
    SelectedItem="{Binding SelectedComputer, Mode=TwoWay}" />
```

6.4 Verbleibendes Code-behind

Das Code-behind bleibt auf WPF-spezifische Aufgaben beschränkt: Ermittlung der per Rechtsklick angeklickten DataGridViewCell, Setzen der aktuellen Zelle und Auslesen des gebundenen Zellwerts. Fach- und Infrastrukturentscheidungen liegen nicht im Fenster.

7. Export, Clipboard und Dialoge

Service	Aufgabe
ExcelExportService	Erzeugt XLSX-Dateien, schreibt Daten, formatiert Datum, Tabelle, Breiten und Zeilenumbruch.
FileDialogService	Öffnet SaveFileDialog und liefert den gewählten Pfad oder null.
ClipboardService	Kopiert Text oder einen tabulatorgetrennten vollständigen Computer-Datensatz.
DialogService	Zeigt Informations-, Warn- und Fehlermeldungen über MessageBox.

Durch die Interfaces kann das ViewModel in Unit-Tests mit Fakes betrieben werden. Es muss weder eine echte Datei erzeugen noch auf die Windows-Zwischenablage oder MessageBox zugreifen.

8. Dependency Injection und Start

8.1 App.xaml

App.xaml enthält kein StartupUri. Das Hauptfenster besitzt keinen parameterlosen Konstruktor und wird deshalb ausschließlich durch den DI-Container erzeugt.

8.2 Registrierung

```
services.AddSingleton<IIpAddressService, DnsIpAddressService>();
services.AddSingleton<IActiveDirectoryService, ActiveDirectoryService>();
services.AddSingleton<IClipboardService, ClipboardService>();
services.AddSingleton<IExcelExportService, ExcelExportService>();
services.AddSingleton<IFileDialogService, FileDialogService>();
services.AddSingleton<IDialogService, DialogService>();
services.AddSingleton<MainWindowViewModel>();
services.AddSingleton<MainWindow>();
```

8.3 Lebenszyklus

OnStartup löst MainWindow über GetRequiredService auf, setzt Application.MainWindow und zeigt das Fenster. OnExit gibt den ServiceProvider frei.

9. Fehlerbehandlung und Sicherheit

9.1 Fehlerbehandlung

- ActiveDirectoryQueryException kapselt LDAP- und Verbindungsfehler mit verständlichen Meldungen.

- OperationCanceledException wird separat behandelt und nicht als technischer Fehler umdeklariert.
- DNS-Fehler führen zu einer leeren IP-Adresse, nicht zum Abbruch der Gesamtabfrage.
- DialogService zentralisiert Benutzerhinweise und Fehlermeldungen.
- Export- und Clipboard-Fehler werden im ViewModel abgefangen und gemeldet.

9.2 Sicherheitsaspekte

- Ausschließlich lesender AD-Zugriff.
- Keine Speicherung von Anmeldeinformationen.
- Verwendung des aktuell angemeldeten Windows-Kontos.
- LDAP Signing und Sealing aktiviert.
- Keine BitLocker-Schlüssel oder sonstigen geheimen Werte.
- Keine automatische Übertragung von Daten an externe Dienste.

10. Unit-Tests, Build und Deployment

10.1 Testprojekt

```
dotnet new xunit -n ADComputerSearchTool.Tests
dotnet sln add ADComputerSearchTool.Tests\ADComputerSearchTool.Tests.csproj
dotnet add ADComputerSearchTool.Tests\ADComputerSearchTool.Tests.csproj reference
ADComputerSearchTool\ADComputerSearchTool.csproj
```

10.2 Testschwerpunkte

Testbereich	Beispiele
LdapFilterHelper	Leere Kriterien, Name, Beschreibung, Aktiv/Deaktiviert und Sonderzeichen-Escaping.
DistinguishedNameHelper	OU-Pfade, CN-Auflösung und maskierte Kommata beziehungsweise Pluszeichen.
ComputerRecord	Abgeleiteter Status und Standardwerte.
MainWindowViewModel	Suche, Reset, Export, Kopieren, Fehler- und Busy-Zustände mit Fake-Services.

10.3 Build und Tests

```
dotnet clean
dotnet restore
dotnet build -c Release
dotnet test --configuration Release
```

10.4 Veröffentlichung

```
dotnet publish ADComputerSearchTool\ADComputerSearchTool.csproj \
  -c Release -r win-x64 --self-contained true \
  -p:PublishSingleFile=true
```

Für eine organisationsweite Verteilung sollten zusätzlich Codesignierung, Versionsnummern, Release Notes und ein kontrollierter Bereitstellungsweg berücksichtigt werden.

11. Wartung und Erweiterung

11.1 Regeln für Änderungen

1. Neue Datenfelder zuerst im Model ergänzen.
2. AD-Attribute ausschließlich im ActiveDirectoryService abfragen und abbilden.
3. Neue technische Funktionen über Interface und Service implementieren.
4. UI-Zustand und Befehle im ViewModel halten.
5. XAML nur über Bindings und Commands anbinden; Code-behind auf echte WPF-Spezialfälle begrenzen.
6. Für Helper und ViewModel-Verhalten Unit-Tests hinzufügen.
7. Nach jeder Änderung Release-Build und vollständigen Funktionstest durchführen.

11.2 Empfohlene Verbesserungen

- CancellationTokenSource und sichtbarer Abbruch-Command.
- Begrenzte parallele DNS-Auflösung mit SemaphoreSlim.
- Microsoft.Extensions.Logging mit rotierenden lokalen Logs.
- ApplicationSettings für Port, Timeouts und Ergebnissgrenze.
- Testabdeckung für ViewModel und Fehlerpfade.
- CI-Pipeline für Build, Test und Release-Artefakte.

Definition of Done

Das Projekt gilt für den aktuellen Funktionsumfang als technisch fertig, wenn der Release-Build ohne Fehler läuft, alle Unit-Tests erfolgreich sind und die fachlichen Funktionen in einem manuellen Abnahmetest bestätigt wurden.